



Examen: Cotizador de Seguros del Hogar con JavaScript



Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva para la cotización de pólizas de seguros del hogar y propiedades**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación debe permitir:

- 1. Consultar tipos de propiedad, zonas de ubicación y costo base por metro cuadrado desde el servidor backend local.
- 2. Renderizar dinámicamente las opciones en los selectores de propiedad y ubicación en el formulario.
- 3. Calcular el costo estimado mensual de la póliza en base a la superficie (m²), el factor de riesgo del tipo de inmueble y el factor de ubicación.
- 4. Persistir las cotizaciones realizadas en el navegador mediante `localStorage`.
- 5. Visualizar el historial de pólizas cotizadas y permitir su limpieza interactiva.



Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/seguros</code> , <code>/api/propiedades</code> , <code>/api/ubicaciones</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de seguros con fetch y async await</code>
#3	Renderizar dinámicamente las opciones de propiedades y ubicaciones en el DOM.	<code>feat(js): renderizar opciones de propiedades y ubicaciones en el dom</code>
#4	Implementar el cálculo del valor de la póliza al enviar el formulario.	<code>feat(js): implementar calculo y cotizacion de poliza de seguro</code>
#5	Persistir las pólizas en <code>localStorage</code> , mostrar el historial y permitir su limpieza con <code>#btnLimpiarHistorial</code> .	<code>feat(js): persistir y gestionar historial de seguros en localStorage</code>



Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/seguros` : Devuelve el objeto completo con `costoBasePorM2`, `propiedades` y `ubicaciones`.
- `GET http://localhost:3000/api/propiedades` : Devuelve el arreglo de tipos de propiedad con sus factores.
- `GET http://localhost:3000/api/ubicaciones` : Devuelve el arreglo de ubicaciones con sus factores.

Para iniciar el servidor backend:

```
npm start
```

2. Fórmula de Cotización de Póliza

$$\text{Valor Póliza} = \text{costoBasePorM2} \times \text{metros2} \times \text{factorPropiedad} \times \text{factorUbicacion}$$

3. Elementos Clave del DOM

- `#propiedad` : `<select>` donde se cargan los tipos de propiedad.
- `#ubicacion` : `<select>` donde se cargan las zonas geográficas.
- `#metros2` : Input numérico para ingresar la superficie.
- `#formSeguros` y `#btnCotizar` : Formulario y botón para calcular y guardar la póliza.
- `#valorPoliza` : Span donde se renderiza el precio estimado.
- `#historialLista` : Lista `<ul>` donde se registran las cotizaciones guardadas.
- `#btnLimpiarHistorial` : Botón para vaciar el historial en `localStorage`.

4. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'seguros_historial'`
- **Estructura:** Arreglo de objetos con `{ propiedad, ubicacion, metros2, valorPoliza, fecha }`.
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.